

APPENDICES

Appendix A - Comparative Analysis of Semantic Type Detection Approaches

This appendix presents an **extended** comparative analysis of representative systems for semantic type detection. **Table A** consolidates the survey of Liu et al. [23] and **updates/extends** it with recent approaches (including **LLM-based metadata** methods) and **this header-centric framework**.

Dimensions covered include system class, header usage, **header-only capability**, knowledge graph integration (DBpedia/Schema.org), benchmark datasets, SemTab setting/participation (including offline comparisons), and key techniques, offering a concise view of recent trends and gaps.

Table A - Extended comparative analysis of semantic type detection approaches, adapted and expanded from Table 1 in Liu et al. [23].

System (Algorithm)	Year	Class	Header Usage	Header-only Capable?	KG	Data Source	SemTab?	Key Technique
Wang et al. [44]	2012	Heuristic, Lookup	Primary	No	Probase	Custom Wikipedia Tables	Baseline	Probase + rules
C ² [19]	2021	Heuristic, Lookup	Probabilistic	No	DBpedia, Wikidata	Limaye, ISWC2017, SemTab 2019, T2D	Winner CTA 2020	Ensemble/statistics
Magic [40]	2021	Heuristic, Lookup	Direct compare	No	DBpedia, Wikidata	SemTab 2021	Participant	INK embeddings
Alobaid et al. [3]	2022	Heuristic, Lookup	Main input	Yes	DBpedia	SemTab 2021, T2D	Participant	String similarity & normalization
TableMiner+ [47]	2017	Heuristic, Iterative	Lexical	No	Freebase	Limaye, IMDB, MusicBrainz	Baseline	Lexical/iterative
CSV2KG [41]	2019	Heuristic, Iterative	Fallback	No	DBpedia	SemTab 2019	Baseline	Heuristic
Mtab [29, 30, 31]	2019	Heuristic, Iterative	Ensemble	No	DBpedia, Wikidata	SemTab 2019–2021	Winner	Lookup, NLP ensemble
Limaye et al. [22]	2010	Feature Engineering	Strong	No	YAGO	Limaye	Baseline	Prob. graphical model
Mulwad et al. [27,28]	2010	Feature Engineering	Limited	No	Wikilogy	Limaye	Baseline	Heuristic, SVM
DAGOBAD Embeddings [6]	2019	Deep Learning, KG Modelling	Central	No	DBpedia, Wikidata	SemTab 2019	Baseline (CTA)	ML+heuristic ensemble
Sherlock [14]	2019	Deep Learning, Table Modelling	Minimal (opt.)	No	DBpedia	T2D, VizNet	Baseline	Deep CNN
Sato [45]	2020	Deep Learning, Table Modelling	No header	No	DBpedia	VizNet	Baseline	TabNet/CRF/BERT
TURL [10]	2020	Deep Learning, Table Modelling	Limited	No	DBpedia	WikiGS, WikiTable, T2D	No	Tabular transformer
Doduo/BERT-CTA [42]	2022	Deep Learning, Table Modelling	Limited	No	-	WikiTable, VizNet	Participant	Fine-tuned BERT (multi-task)
RECA Sun et al [39]	2023	Deep Learning, Table Modelling	Limited	No	DBpedia	T2Dv2, SemTab, Wikipedia	No	Related table context + features
ADWAN	2024	LLM-based, Metadata	Prompts LLM	Yes	Dbpedia, Schema.org	SemTab2024 Metadata Only track	Winner	RAG + CoT + Self-Consistency + RRF
CVA/Metalinker(2024)	2024	LLM-based, Metadata	Main input	Yes	Dbpedia, Schema.org	SemTab2024 Metadata Only track	Participant	Zero-shot LLMs + RAG + SemanticBERT
Anonymized [37]	2024	Heuristic, Lookup	Exclusive header	Yes	-	UCI	No	Dynamic, feedback-driven, hybrid fallback
This paper	2026	Heuristic, Lookup	Exclusive header dynamic	Yes	Dbpedia, Schema.org	Kaggle, VizNet, Sato, UCI, Prague, SemTab 2024 Metadata Only track	2024 Offline comparison	Rule-based + feedback/ML fallback

Notes: (i) “Header-only” indicates operation without cell values; (ii) “SemTab note” distinguishes official participation from **offline** evaluations; (iii) rows and sources were cross-checked against cited papers; the curation criteria and change log are available in [38] for reproducibility.

Appendix B - Machine Learning Details (Attribute-BasedMLSemanticTypeDetection. ipynb)

This appendix documents the optional ML classifier pipeline, which complements the rule-based stage by producing per-column format predictions using header-only evidence. It details preprocessing, feature extraction, model training, and evaluation, and accompanies **Algorithm 1**, which specifies where the ML classifier is invoked within the semantic type detection workflow.

For reproducibility, all configuration parameters, including random seed, data-split policy, cross-validation, hyperparameter grids, and persistence of trained models, are listed at the end of this appendix.

Scope note: In this study, the ML classifier was run only on the first 100 data sources (50 UCI and 50 Prague) with approximately 1,400 columns. All other experiments used the rule-based pipeline alone.

Preprocessing and Feature Engineering.

Column headers and descriptions are cleaned through normalization, abbreviation replacement, and camel-case splitting (see **Section 3.3**). The resulting textual data is vectorised using a **TF-IDF model** [33] built from a combined vocabulary of **DBHeaders** [9] and the **Formats dictionary**. In parallel, **dictionary-derived SourceKeywords** are extracted to record which terms triggered candidate formats, supporting explainability, consistency checks, and enrichment when NIL cases are logged.

Addressing Class Imbalance.

Semantic type classification often suffers from skewed distributions, with some types appearing far more frequently than others. To mitigate this imbalance, we employ the **Synthetic Minority Over-sampling Technique (SMOTE)**, which generates synthetic minority class samples to enhance generalisation while reducing the risk of overfitting.

Model Selection and Hyperparameter Optimization.

Multiple classifiers are evaluated, including **Random Forest**, **Logistic Regression**, **Gradient Boosting**, **K-Nearest Neighbours**, and **LinearSVC**. Hyperparameters are tuned via **GridSearchCV** with stratified cross-validation to ensure fair optimization across imbalanced classes.

Evaluation and Interpretability.

Models are assessed using precision, recall, and F1-score on held-out test data. Feature importance analysis is employed to interpret classification behaviour and identify influential terms.

Model Persistence.

All trained classifiers, together with the fitted TF-IDF vectorizer, are saved to support reproducibility and allow flexible deployment scenarios.

Algorithm 1 – Attribute-Based ML Semantic Type Detection
(applied systematically to all non-ID columns; dictionaries provide features and reconciliation)

Inputs:

dataInfo (dataset/table metadata and columns); formatDict (Formats Dictionary); abbrDict (Abbreviations Dictionary); dbHeadersInfo (DB headers corpus); classifiers ({RandomForest, LogisticRegression, GradientBoosting, KNN, LinearSVC}); tfidfVectorizer (trained on Formats+DBHeaders vocabulary).

Output:

analysedColumns_models (per-column predictions per model + aggregate, confidence, NIL/flags, review logs).

```

1  function SemanticTypeDetect(dataInfo, formatDict,
  abbrDict, dbHeadersInfo,
2      classifiers, tfidfVectorizer, confidenceThreshold)
3      columns ← ExtractAndCleanColumns(dataInfo) // trim,
  normalise case/punctuation, etc.
4      for each column in columns do
5          // Deterministic ID assignment (DB context)
6          if column.belongsToDatabase then
7              pkfk ← GetPrimaryForeignKeyInfo(column, dataInfo)
8              if pkfk.isPK or pkfk.isFK then
9                  column.semanticType ← "ID column"
10                 column.mlConfidence ← 1.0
11                 SavePerModelPredictions(column, {}) // optional
  empty per-model record
12                 continue
13             end if
14         end if
15         // Dictionary-derived FEATURES (no hard gate)
16         expanded ← ReplaceAbbreviations(column.cleanedName, abbrDict)
17         expanded ← SplitCamelCase(expanded)
18         sourceKeywords ← ExtractKeywords(expanded,
  formatDict) // e.g., ColumnKeyword, DescriptionKeyword
19         // Compose text input and vectorise
20         if column.description ≠ ∅ then
21             inputText ← Concatenate(expanded, " ",
  column.description)
22         else
23             inputText ← expanded
24         end if
25         X ← tfidfVectorizer.transform([inputText])
26         // Predict with all classifiers; collect per-model outputs
27         preds ← {} // model → {label,
  confidence}
28         for each clf in classifiers do
29             y_hat ← clf.predict(X)
30             p_max ← PredictConfidence(clf, X, y_hat) //
  calibrated prob / softmax / decision fn
31             preds[clf.name] ← (y_hat, p_max)
32         end for
33         // Aggregate predictions (e.g., weighted by validation
  F1)
34         (aggType, aggConf) ← AggregatePredictions(preds,
  weights=ValidationScores(classifiers))
35         column.semanticType ← aggType
36         column.mlConfidence ← aggConf

```

```

37 // Reconciliation & review signals
38 column.ruleSignals ← sourceKeywords //
retained for explainability
39 column.consistencyFlag ←
Consistency(ruleHint=BestRuleHint(sourceKeywords),
40 mlType=aggType) // AGREE /
DISAGREE / NONE
41 // Low-confidence handling → NIL + enrichment queue
42 if aggConf < confidenceThreshold then
43 column.semanticType ← "NIL"
44 LogForReview(column, sourceKeywords, TopK(preds,
k=3)) // store inputs + top candidates
45 AppendToEnrichmentQueue(sourceKeywords,
column) // drive dictionary/abbr updates
46 end if
47 SavePerModelPredictions(column, preds) //
persist all models' outputs
48 end for
49 return analysedColumns_models(columns)
50 end function

```

Reproducibility Details

We fix the random seed to 42 across all steps. Data are split with 80/20 stratified hold-out (train_test_split(test_size=0.2, stratify=labels)), and model selection uses GridSearchCV with 3-fold CV (cv=3, n_jobs=-1).

Imbalance handling: We apply SMOTE with k_neighbors=nn (as in the released code) and a task-specific sampling strategy.

Feature extraction: a custom **TF-IDF** vectorizer built from the **union** of **DBHeaders** tokens and the **Formats** dictionary (as normalised in §2.3).

Evaluated models & grids:

RandomForest (n_estimators ∈ {100,200}, max_depth ∈ {None,30}, min_samples_split ∈ {2,5}), **GradientBoosting**

(n_estimators=100, learning_rate=0.1, max_depth=3),

LogisticRegression (C ∈ {0.1,1}, solver='lbfgs', max_iter=20000),

KNN (n_neighbors ∈ {3,5}, weights ∈ {uniform,distance}),

LinearSVC (C ∈ {0.1,1}, max_iter=20000, dual='auto').

Persistence: the selected model per classifier and the TF-IDF vectorizer are exported via **joblib** to support exact reproduction.

Repository: code, dictionaries (Formats/Abbreviations), range tables, and the FinalFormat↔KG mapping tables are released in [38].

Results on 100 Data Sources

We evaluated five classifiers (Random Forest, Gradient Boosting, K-Nearest Neighbors, Logistic Regression, and Linear SVC) using the preprocessing and balancing strategy described in Section 3.3 and in this appendix. Class imbalance was particularly relevant for minority semantic types (e.g., URLformat, Time), where we applied SMOTE to improve recall. Models were tuned with GridSearchCV and assessed by

precision, recall, and F1-score on an 80–20 split. The data sources used were the 50 UCI datasets and the 50 Prague Relational Data Tables.

Per-format results for each classifier (precision, recall, and F1-score) are reported in the supplementary plots corresponding to Figures A-C.

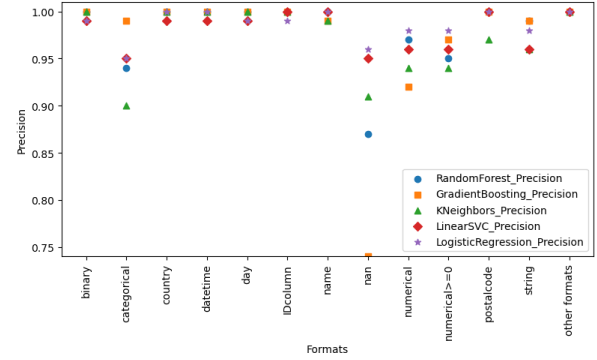


Figure A - Precision for Formats on 5 ML Models

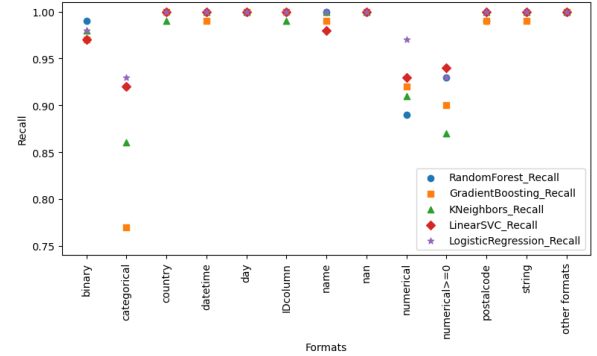


Figure B - Recall for Formats on 5 ML Models

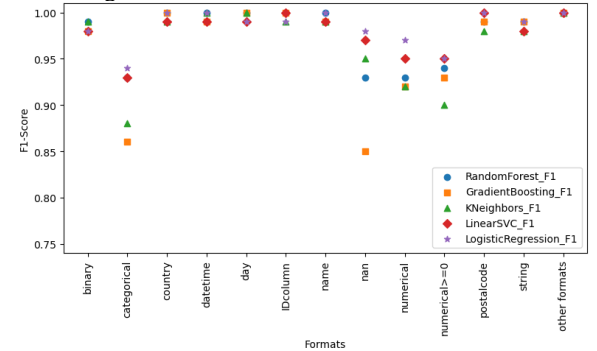


Figure C - F1-Score for Formats on 5 ML Models

Categorical columns showed the most variability across models. At the same time, the distinction between Numerical and Numerical>=0 remained a frequent source of confusion. Gradient Boosting performed worst on ambiguous or NIL cases. In contrast, Logistic Regression and LinearSVC were more robust. Several formats consistently achieved perfect precision, recall, and F1-score across all classifiers, including Email, URL, Age, City, Country, Date, and Phone. Figure D summarises average performance across the five models.

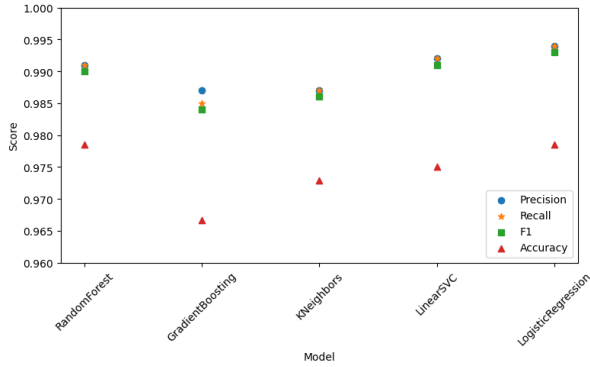


Figure D - Average Metrics on 5 ML Models

The near-perfect results observed in these figures arise from the controlled scope of the machine-learning subset rather than overfitting. The evaluation covered only about 1,400 columns, a scale small enough to allow full manual verification of the training and test labels, ensuring almost noise-free supervision. This explains why several semantic types (e.g., Email, URL, Date, City) achieved an F1 score of approximately 1 across all classifiers.

In contrast, the large-scale rule-based analysis, spanning more than 118,000 columns that the ML models did not process, revealed around 13% NIL cases, reflecting unresolved or ambiguous types under real-world variability. Consequently, the ML results should be interpreted as an upper-bound performance under perfect annotation, while the large-scale NIL proportion represents the lower-bound realism of open-domain data. This distinction clarifies that the high metrics result from dataset certainty, not optimistic bias or information leakage.

Appendix C - FinalFormat coverage in Universe B (Kaggle + VizNet)

This appendix reports the coverage of the 39 **FinalFormats** in **Universe B** (Kaggle + VizNet), focusing on (i) the 15 core FinalFormats that deterministically cover the 76 Sherlock/Sato labels and (ii) the additional 24 FinalFormats that occur beyond Sherlock/Sato, together with the NIL category. Percentages are computed over all 118,639 columns in **Universe B**.

Core FinalFormats covering the Sherlock/Sato label space

Concretely, **only 15 core FinalFormats (Table B)** deterministically cover the *Sherlock/Sato* label space: **all 76 Sato formats map many-to-one into these 15 types**, and the two additional *Sherlock* labels (*File size* and *Team name*) do not appear in Sato's dataset. This consolidation preserves distinctions that matter for data quality (e.g., **IDcolumn**, **date**, **postalcode**) while avoiding unnecessary granularity by merging near-duplicate labels (e.g., creator-like labels such as *Artist*, *Creator*, *Director*, *Jockey*, *Organisation*, *Publisher* → **name**; affiliation/brand-style variants → **categorical**).

Within **Universe B**, these 15 core FinalFormats account for **≈80%** of all columns.

The distribution is **skewed** towards a handful of operationally critical types: *categorical* (19.41%), *numerical>=0* (18.05%), *IDcolumn* (11.10%), *string* (9.50%), and *numerical* (7.64%), highlighting where robust handling mostly affects practice.

Table B - 15 FinalFormats that map all Sherlock/Sato Formats

#	FinalFormats	SherlockFormats	%
1	age	Age	1.04%
2	categorical	Affiliate, Affiliation, Album, Brand, Category, Class, Classification, Club, Code, Collection, Company, Continent, Education, Family, Format, Gender, Genre, Industry, Language, Location, Manufacturer, Origin, Position, Product, Ranking, Region, Religion, Result, Service, Sex, Species, Status, Symbol, Team, Type	19.41%
3	city	Birth place, City	0.85%
4	country	Country, Nationality	0.83%
5	date	Birth date	2.59%
6	day	Day	0.25%
7	IDcolumn	ISBN	11.10%
8	money	Currency	1.10%
9	name	Artist, Creator, Director, Jockey, Name, Operator, Organisation, Owner, Person, Publisher, Team name	5.45%
10	numerical	Credit, Elevation, Range	7.64%
11	numerical>=0	Area, Capacity, Depth, Duration, File size, Grades, Rank, Sales, Weight	18.05%
12	state	State	0.50%
13	street	Address	0.28%
14	string	Command, Component, Description, Notes, Order, Plays, Requirement	9.50%
15	year	Year	1.72%
SUB TOTAL			80.31%

Additional FinalFormats beyond Sherlock/Sato

Beyond *Sherlock/Sato*, we cover an additional **24 formats (Table C)**, notably **bounded numericals** (e.g., *percentage*, chemical related (*ph*, *acidity*, *alkalinity*), geographically related (*latitude/longitude*), health related (*heartrate*, *bloodpressure*), enabling finer-grained DQI checks that traditional taxonomies omit. These add up to around **6%**, **totalling over 7,000 columns in this universe, which is quite considerable**.

In **Universe B**, **NIL = 13.39%** (thus **Valid = 86.61% = ~87%**) over **118,639** headers. Here, *NIL* denotes columns with **no match** after normalization and abbreviation expansion. This rate is low, given the corpus size and heterogeneity, and it quantifies the remaining long tail of noisy or novel headers.

Table C - Additional Formats not considered by Sherlock/Sato

#	FinalFormat	%	#	FinalFormat	%
1	binary	1.53	14	datetime	0.17
2	acidity	0.03	15	E-mailformat	0.13
3	alkalinity	0.00	16	IPformat	0.12
4	angle	0.03	17	modelName	0.01
5	bloodpressure	0.11	18	month	0.14
6	heartrate	0.18	19	NIL	13.39%
7	hour	0.04	20	phone	0.17
8	latitude	0.14	21	postalcode	0.10
9	longitude	0.14	22	time	0.85
10	normalized	0.05	23	URLformat	0.34
11	percentage	1.89	24	week	0.07
12	ph	0.02	25	weekday	0.05
13	saltiness	0.00	SUB TOTAL		19.69%

Appendix D - Bounded Numerical Types

This appendix lists the bounded numerical formats adopted throughout the rule-based and data-quality pipelines. Each of these formats defines a numeric attribute whose valid range is defined a priori, enabling automatic detection of out-of-range and physically impossible values during data quality assessment (**Section 3.7**). The bounds shown below were derived from domain standards and dataset statistics and are applied uniformly across all evaluated data sources. These limits are **not fixed thresholds**; they are intentionally subjective and can be easily adjusted to reflect alternative domain assumptions or updated business rules, for example, different accepted ranges for FinalFormats *year* or *age*.

Table D - Limits of Bounded Numerical formats

Bounded Numerical	Minimum	Maximum	Bounded Numerical	Minimum	Maximum
acidity	0	7	normalized	0	1
age	0	130	numerical between 0 and 360	0	360
alkalinity	7	14	numerical between 0 and 60	0	60
bloodpressure	0	250	percentage	0	100
day	1	366	ph	0	14
heartrate	40	200	saltiness	0	40
hour	0	24	tannins	0	100
latitude	-90	90	week	1	53
longitude	-180	180	year	1800	2100

Appendix E - Top 100 list of NIL tokens ordered by amount

Here is a snapshot of the top 100 terms that appear in the columns that had not been assigned any format. Observe a list of tokens that are not easily associated with any formats. Many are very short or contain symbols or numbers (e.g., *no*, *so*, *ga*, *po*, *+*, *1*, *2*):

Table E - Top 40 NIL Tokens

#	CleanedColumn	Count	#	CleanedColumn	Count
1	no	278	21	sa	30
2	so	149	22	f	30
3	Nil	98	23	wp	28
4	min	80	24	gd	26
5	ga	73	25	st	25
6	sv	73	26	rc	24
7	+	71	27	off	22
8	to	68	28	s no	21
9	lg	64	29	pl	21
10	s	60	30	m	21
11	er	56	31	maf	21
12	gf	49	32		2
13	cg	49	33		1
14	sf	45	34	1b	19
15	from	40	35	c	19
16	pp	36	36	ch	19
17	dp	34	37	zero	19
18	po	33	38	pb	19
19	b	32	39	yc	18
20	ba	32	40	tav	18

Table F - Bottom 60 NIL Tokens

#	CleanedColumn	Count	#	CleanedColumn	Count
41	shg	18	71	replies	13
42		3	72	mat	13
43	fc	17	73	prd	13
44	bf	17	74	may	13
45	x	17	75	toi	13
46	out	17	76	rf	13
47	sl no	16	77	ot	13
48	bs	16	78	ra	13
49	new	16	79	away	13
50	lvl	16	80	str	12
51	streak	16	81	warp	12
52	for	15	82	generosity	12
53	us	15	83	stds	12
54	medal	15	84	winnings	12
55	otl	15	85	thailand	12
56	designation	15	86	bpg	12
57	lost	14	87	guild	12
58	fs	14	88	arr	12
59	vote	14	89	bop	12
60	chest	14	90	wpa	12
61	gw	14	91	pt	12
62	ta	14	92	joined	12
63	extension	14	93	realm	11
64	stats	14	94	station	11
65	ff	14	95	delta	11
66	nat	14	96	@bat	11
67	tries	13	97	previous	11
68	tdp	13	98	adult	11
69	dep	13	99	subcategory	11
70	defense	13	100	msrp	11

Observe that in **Table F** above, many terms can be associated with one of the 39 **FinalFormats**.

Appendix F - Preprocessing and Semantic Type Assignment: Detailed Workflow

Our semantic type detection pipeline employs a comprehensive preprocessing and mapping approach that transforms raw column headers into interpretable semantic annotations via the **FinalFormat** type system. This process involves a series of systematic steps, each designed to maximize robustness, flexibility, and explainability.

Our semantic type detection pipeline executes the following steps, closely mirroring the production implementation:

1. Initial Parsing and Metadata Extraction:

- Extract numeric prefixes (e.g., “1.ID”) for traceability.
- Parse column names by splitting at colons, slashes, or parentheses to isolate core terms.

- Extract free-text descriptions from split points, or from parentheses if none exist, ensuring use of all available metadata.

2. Header Normalization:

- Convert all header tokens to lowercase.
- Remove or replace punctuation and special characters (underscores, hyphens, asterisks).
- Split compound words using spaces, dots, underscores, hyphens, or camelCase patterns (e.g., “AvgPitStopTime” → “avg pit stop time”).
- Explicitly tokenize symbols (e.g., “%”) for percentage detection.
- Match plural and singular forms (e.g., “chlorides” → “chloride”).

3. Abbreviation and Variant Expansion

- Replace abbreviations using a dictionary of over 1000 entries (e.g., DOB → date of birth; bp → blood pressure).
- Apply regular expressions to tokenize and substitute complex patterns, including units (mg/dL, kg/m²), and non-alphabetic tokens like °C, cm², etc.

4. SourceKeywords Extraction

- Identify key tokens (SourceKeywords) that capture semantic meaning (e.g., “age”, “date”, “amount”).
- Scan descriptions for known tokens, including “has”, “is”, or numeric/temporal keywords.
- Use primary/foreign key metadata to override assignments where appropriate.

5. Semantic Type Assignment (Rule-Based and ML classifier Logic)

- Match tokens to a curated dictionary (2,800 mappings) to assign one of 39 FinalFormat types.
- Apply priority logic:
 - “binary” overrides all if “has”, “is”, or “or not” is present.
 - “percentage” takes precedence if “%” or related terms are detected.
 - Hierarchical rules resolve conflicts between categorical, numerical, and domain types.
 - Special logic for “name” columns in certain contexts (city, state, etc.).
- If no match is found:
 - Retry abbreviation expansion and check individual tokens (with plural handling).
 - If still unresolved, use ML-based prediction.

6. Ambiguity Resolution

- Use ML classifier (Random Forest, Logistic Regression, etc.) with weighted TF-IDF features.

- Assign confidence scores to all assignments for human-in-the-loop review.

7. Knowledge Graph Annotation

- Perform syntactic and semantic matching (FastText) between normalized headers and KG property/class labels.
- Use substring and similarity logic for best match selection (DBpedia, Schema.org).

8. Final Type Selection and Logging

- Consolidate assignments if column and description differ, prioritizing by semantic importance (e.g., prefer “datetime”, “IDcolumn”, etc.).
- Attribute assignments to specific source keywords for explainability.
- Assign “Nil” and log cases for manual review if no match is found.

Our semantic type detection pipeline employs a comprehensive sequence of normalization, expansion, and mapping steps that transform raw column headers into interpretable semantic annotations via the FinalFormat type system. The workflow (detailed above) enables robust, transparent, and flexible type assignment across real-world tabular data.

Practical Examples of Semantic Type Assignment

Table G below demonstrates, on a large and diverse set of real columns, how our pipeline’s dictionary logic, abbreviation expansion, and contextual rules deliver precise, explainable semantic types.

Key Interpretation Patterns and Highlights

Binary Columns:

Binary-type columns are often signaled by common words such as “has”, “is”, or phrases like “or not”. The pipeline detects these cues and classifies such columns under the binary FinalFormat, ensuring boolean attributes are annotated correctly even when explicit True/False values are absent.

Disambiguating Categorical Types:

Columns containing the word “range” (e.g., “price range”, “date range”) are accurately identified as categorical, despite potentially misleading co-occurring terms like “date” or “price” that might otherwise point toward temporal or financial types. This illustrates the nuanced, context-aware mapping the system achieves via dictionary-based disambiguation.

Geographic and Demographic Mapping:

Headers with words like “nationality”, “province”, or “capital” are consistently mapped to country, state, or city FinalFormats.

Synonym handling ensures that “ProvinceName” is linked to FinalFormat state, and “capital name” to FinalFormat city, even if these terms don’t match dictionary entries exactly.

Temporal and Timestamp Recognition:

Fields such as “updated”, “day of week”, and “timestamp” are mapped to their appropriate FinalFormats:

“updated” → date
 “request timestamp” → datetime
 “day of week” → weekday

This allows precise differentiation between temporal and generic date columns.

Financial and Monetary Attributes:

Lexical cues like “cash”, “currency”, “usd”, and “price” reliably trigger the assignment to the money FinalFormat, enabling wide coverage of financial information.

Communication and Identifier Columns:

Terms like “fax”, “cellular phone”, and “author channel id” are normalized to FinalFormat phone and FinalFormat IDcolumn, respectively, with common abbreviations (e.g., “nr” for number) mapped to FinalFormat numerical.

Scientific and Measurement Values:

Abbreviations and specialized terms, such as “wgt avg” (weight average) or “trestbps” (blood pressure), are interpreted using expansion rules, with assignments to FinalFormat numerical ≥ 0 and FinalFormat bloodpressure, reflecting the system’s capacity to handle both generic and domain-specific semantics.

Web and Address Columns:

Patterns like “web address”, “link”, or “href” result in URLformat annotations, unifying various forms of URL-related fields.

Postal and address information, including “zipcode”, “provider zip code”, and “purchase address”, is reliably mapped to FinalFormat postalcode or FinalFormat street.

String and Free-Text Columns:

Terms such as “abstract”, “desc 1”, and “spec abstract” are interpreted as free text and assigned the FinalFormat string.

Other Notable Patterns:

- Medical/Health: Medical headers like “bp” or “trestbps” are mapped to FinalFormat bloodpressure.
- Percentage: Any header containing the “%” symbol or “percent” is assigned FinalFormat percentage.
- Temporal granularity: “Month”, “hour”, “week number”, and “year” are all mapped to their corresponding temporal FinalFormats.

Through these patterns, **Table G** provides a transparent view of our pipeline’s ability to handle abbreviation, synonymy, compound phrases, and real-world header messiness, delivering precise, actionable semantic types for data quality assessment.

These examples illustrate how the system transforms raw, often noisy or abbreviated, column headers into precise semantic annotations. By combining normalization, robust abbreviation expansion, and hierarchical rule logic, our pipeline ensures reliable type assignment even in the face of ambiguity, domain-specific terminology, or non-standard header formats. This approach not only supports accurate data profiling but also underpins the effectiveness of our automated data quality assessment framework.

Table G

Original Header	Normalized	SourceKeywords	FinalFormat
Members with age 5 - 17 years old	members with age 5 17 years old	age	age
Ankle_Ground_Angle	ankle ground angle	angle	angle
has_birth_date	has birth date	has	binary
IsBasedOnRealStory	is based on real story	is	binary
BookedHotelOrNot	booked hotel or not	or not	binary
bp	bp	blood pressure	bloodpressure
trestbps	trestbps	blood pressure	bloodpressure
Reported Influenza Activity	reported influenza activity	activity	categorical
Aircraft Manufacturer	aircraft manufacturer	aircraft	categorical
Pclass_1	pclass 1	class	categorical
ATTEND_DEPT	attend dept	department	categorical
Date Range*	date range*	range	categorical
price_range	price range	range	categorical
Father's Birth Place	father's birth place	birth place	city
CapitalName	capital name	capital	city
Author, Country	author, country	country	country
team_one_player_one_nationality	team one player one nationality	nationality	country
BIRTHDATE	birthdate	birthdate	date
dtRef	dt ref	date	date
Data Last Updated	data last updated	updated	date
Order Date and Time	order date and time	date and time	datetime
Request timestamp	request timestamp	timestamp	datetime
school_day	school day	day	day
FLAG_EMAIL	flag email	email	E-mailformat
order_hour_of_day	order hour of day	hour	hour
authorChannelId	author channel id	id	IDcolumn
Nameserver IP Address	nameserver ip address	ip	IPformat
src_ip_country_code	src ip country code	ip	IPformat
vehicle_gps_latitude	vehicle gps latitude	latitude	latitude
Delivery_location_longitude	delivery location longitude	longitude	longitude
Cash amount	cash amount	cash	money
discount_price_currency	discount price currency	currency	money
cons.price.idx	cons price idx	price	money
Values in Billions USD	values in billions usd	usd	money
Date_Of_Death_Month	date of death month	month	month
Head Coach	head coach	coach	name
Artistic Director	artistic director	director	name
fullname words	fullname words	fullname	name
LASTNAME	lastname	lastname	name
Person Baptised	person baptised	person	name
Allowed Amount	allowed amount	amount	numerical
Trip_Distance_km	trip distance km	distance	numerical
Ground Elevation	ground elevation	elevation	numerical
Flug-Nr.	flug nr	number	numerical
Deforestation Area Ha	deforestation area ha	area	numerical >=0
free_throw_attempts	free throw attempts	attempts	numerical >=0
ViewDepth	view depth	depth	numerical >=0
DirectoryEntryImportSize	directory entry import size	size	numerical >=0
Wgt. Avg.	wgt avg	weight	numerical >=0
YearsInCurrentRole	years in current role	years	numerical >=0
% of Total Supply Owned	% of total supply owned	%	percentage
Percent of State employment	percent of state employment	percent	percentage
Number of Cellular phone	number of cellular phone	cellular phone	phone
Fax Numbers	fax numbers	fax	phone
Customer_Postal_Code	customer postal code	postal code	postalcode
Provider Zip Code	provider zip code	zip code	postalcode
ProvinceName	province name	province	state
Purchase Address	purchase address	address	street
Spec. abstract	spec abstract	abstract	string
desc_1	desc 1	description	string
AvgPitStopTime	avg pit stop time	time	time
goods-title-link-jump href	goods title link jump href	href	URLformat
Free Download Link	free download link	link	URLformat
Web Address	web address	web address	URLformat
arrival_date_week_number	arrival date week number	week number	week
Day of Week	day of week	day of week	weekday
athlete_year_birth	athlete year birth	year	year

Appendix G - Full SemTab 2024 Metadata-to-KG Track [12] Analysis

G.1 Methods (Human Review Protocol)

One author, **blinded to official GT labels and to all cell values**, adjudicated each prediction using only the **column header** and public ontology documentation (definitions and official redirects). Each case was assigned to one of four categories:

- **C1 (GT-CONF)**: Predicted URI equals the GT URI or an **official redirect to the same canonical entity**. (*None were found beyond the official script; C1 contributes 0 in our audit.*)
- **C2 (GT-REFINE)**: Predicted URI is **not** the GT entity but is a **more appropriate** entity/property for the header (change of granularity/scope/type that better reflects the header). We submit these as **candidate GT issues**.
- **C3 (ALT)**: Predicted URI is **not** the GT entity and **not necessarily better**, but remains **plausible** under header-only interpretation (synonym/alias/generalization).
- **C4 (INC)**: Remaining cases (NIL, non-English unresolved, unit/numeric-only, ambiguous, lexical confusion, etc.).

We report raw counts and, where space permits, **95% binomial Wilson confidence intervals** for proportions.

G.2 Results Overview (n = 141)

Official (GT-strict). Using exact URI equality for the same canonical entity (no manual flags):

- **Hit@1 (strict)**: 63/141 $\approx$ 0.45 (95% CI $\approx$ [0.36, 0.53])
- **Hit@5 (strict)**: 67/141 $\approx$ 0.48 (95% CI $\approx$ [0.40, 0.56])

Audit reclassification (ours only; header-only rubric). From the adjudicated set:

- **C1 (GT-CONF, additional)**: 0 (*no extra same-entity corrections beyond strict*)
- **C2 (GT-REFINE)**: 43/141 (30.5%)
- **C3 (ALT)**: 6/141 (4.0%)
- **C4 (INC)**: 24/141 (17.0%)

Diagnostic Hit@1 (ours only; not for cross-system ranking).

- **B1: Strict**: 63/141 = 0.447 ($\approx$ 0.45)
- **B2: B1 + C2 (GT-REFINE)**: (63+43)/141 = 106/141 = 0.752 (95% CI $\approx$ [0.67, 0.81])
- **B3: B2 + C3 (ALT)**: (106+6)/141 = 112/141 = 0.794 (95% CI $\approx$ [0.72, 0.85])

Diagnostic Hit@5 (ours only; header-only audit)

Starting from the strict Top-5 match count (67/141 = 0.475), we apply the same cumulative reconciliation used for Hit@1:

- **B1 — Strict**: 67/141 = 0.475 (95% CI $\approx$ [0.40, 0.56])
- **B2 — B1 + C2 (GT-REFINE)**: (67 + 43)/141 = 110/141 = 0.781 (95% CI $\approx$ [0.71, 0.85])
- **B3 — B2 + C3 (ALT)**: (110 + 6)/141 = 116/141 = 0.823 (95% CI $\approx$ [0.76, 0.88])

Notes.

1. **C1 (GT-CONF)** contributed 0 cases; therefore, Panel B starts from the strict baseline.
2. These figures are **diagnostic** and apply to **our outputs only**; they are **not used for cross-system ranking**.
3. Under this diagnostic view, **C2/C3 are counted as correct** for Top-5 irrespective of whether the GT URI appears in the model's Top-5 list, because the human audit judged them ontology-consistent and header-appropriate under the rubric.

G.3 Tables

Table H - Columns Identified as Correct After Human Review (43 items)

Column values	GT Annotation	Our Annotation	Items	Justification
Year	releaseDate	year	20	More accurate general temporal reference
ISO(2)/ISO(3)	iso31661Code	isoCode	2	The general ISO code is semantically suitable
Length	duration	length	1	Physical dimension is more appropriate
Government	governmentType	government	4	Precisely matches the semantic context
State	location	state	4	Accurate geopolitical specificity
GDP	grossDomesticProduct	gdpPerCapita	1	Contextually appropriate economic indicator
Alphabeticcode	currencyCode	code	4	Correct generalization without context
GrantingInstitution	almaMater	institution	1	Broader semantic applicability
Height(ft), HEIGHTINMETERS	elevation	height	5	More precise semantic choice for non-geographical context
Board	owner	board	1	Semantically matches governance/advisory body
		TOTAL	43	30.5%

Table I - Contextually Appropriate Columns (6 items)

Column values	GT Annotation	Our Annotation	Items	Justification
Size	fileSize	collectionSize	1	Semantically appropriate given context
Label	developer	distributingLabel	1	Contextually plausible alternative
System	computingPlatform	systemRequirements	3	Ambiguous header, plausible semantic choice
watergauge	elevation	water	1	Contextual understanding of water measurement
		TOTAL	6	4%

Table J - Incorrect Columns (24 items)

Column values	GT Annotation	Our Annotation	Items	Issue
Basincountries, Staat(en)	country	Nil	2	Multilingual/compound words issues
8.848, Feets, Meters, H?he	elevation	Nil	5	Numeric/unit headers lacking context
GNI, GDPnominal(US\$M)	giniCoefficient, grossDomesticProduct	Nil	2	Abbreviations and economic indicators
Ashton-under-Lyne, Editeur, Endroit, Stadt	location, owner	Nil	5	Non-English terms are ambiguous without context
Europe, Japan, Rel.	releaseDate	Nil	5	Contextual mismatch with GT annotation
NorthAmerica	releaseDate	northWestPlace	1	Contextual mismatch with GT annotation
Rationale	knownFor	ratio	2	Semantic misunderstanding due to lexical similarity
1T	iataAirlineCode	Nil	1	headers lacking context
Max.-Tiefe-(m)	depth	max	1	headers lacking context
		TOTAL	24	17%

G.4 Successes and Challenges (Header-Only CTA)

Scope. This section interprets results under the **header-only** constraint. It uses the audit rubric from §G.1: **C.1 GT-Confirmed** (same canonical entity), **C.2 GT-Refinement** (author-proposed replacement for GT), **C.3 Ontology-Consistent Alternative** (plausible but not GT), and **C.4 Incorrect** (incl. NIL/ambiguous/multilingual/unit-only).

G.4.1 Success cases (C.1 / C.3)

A substantial fraction of columns with clear headers are annotated reliably:

- **Examples (C.1 GT-Confirmed where applicable).** Headers such as “Government”, “Year”, and “Height(ft)” tend to map to intuitive DBpedia targets. When the **same canonical URI** as GT is reached (i.e., an identical URI or an official redirect), these are counted under **C.1**.
- **Plausible alternatives (C.3).** In some cases, the model selects ontology-consistent alternatives that remain defensible under header-only semantics, e.g., elevation → height for non-geographic contexts, developer → distributingLabel for “Label”, or computingPlatform → systemRequirements for “System”. These are recorded as **C.3** because they are plausible yet **not** the GT entity.

Takeaway. Under the header-only premise, the system is robust to minor label differences and often converges to semantically sound choices even when GT uses a different—but related—entity/property.

G.4.2 Error and challenge cases (C.4)

Most failures arise from classes of headers that are inherently fragile under header-only interpretation:

- **Multilingual/compound or abbreviated headers.** E.g., “Basincountries”, “Staat(en)”, “Editeur”, “Rel.”
- **Unit-only or numeric-only headers.** E.g., “8.848”, “Feets”, “Meters”, “Max.-Tiefe-(m)”.
- **Code/identifier without scope.** E.g., “1T” (IATA airline code).
- **Ambiguous geography vs. temporal GT.** E.g., “Europe”, “Japan”, “NorthAmerica” marked as releaseDate in GT; such mappings appear to rely on **cell values**, not headers.

We record these under **C.4 Incorrect** (including **NIL** when the system declines to guess). In header-only evaluation, **NIL** can be a desirable behaviour for high-precision operation.

G.5 Human-Informed Diagnostic of GT Inconsistencies (C.2 / C.3)

Motivation. Strict GT requires the predicted URI to match the **same canonical entity** as the ground truth. However, benchmarks may encode **granularity** or **context** that is not present in column headers. We therefore performed a blinded, header-only audit to separate **(i)** genuine model errors from **(ii)** GT idiosyncrasies.

G.5.1 GT-Refinement (C.2 author-proposed replacements)

We identified cases where the model’s URI appears **more appropriate** for the header than the GT entity (candidate GT issues). Representative examples:

- **Year:** releaseDate → year (n=20). The header denotes a general temporal attribute; releaseDate presupposes an event/product.
- **ISO(2)/ISO(3):** iso31661Code → isoCode (n=2). The header does not evidence the 3166-1 scope; a scope-neutral ISO code is header-faithful.
- **Length:** duration → length (n=1). Physical extent vs. time interval.
- **Government:** governmentType → government (n=4). Institution vs. classification attribute.
- **State:** location → state (n=4). Geopolitical specificity.
- **Alphabeticcode:** currencyCode → code (n=4). General “code” without a currency scope in the header.
- **GrantingInstitution:** almaMater → institution (n=1). Granting body vs. alumni relations.
- **Height(ft), HEIGHTINMETERS:** elevation → height (n=5). Non-geographical height signalled by the header.
- **Board:** owner → board (n=1). Governance/advisory body vs. ownership.

Classification. All such cases are recorded as **C.2 GT-Refinement** (not counted as “Correct” under strict GT), and fed into **Panel B** diagnostics to quantify potential GT noise in header-only settings.

G.5.2 Ontology-Consistent Alternatives (C.3)

We also found plausible alternatives that are defensible under header-only semantics, but without claiming they “correct” GT. Examples include “Size: fileSize → collectionSize”, “System: computingPlatform → systemRequirements”, and “watergauge: elevation → water”. These are recorded as **C.3**.

Diagnostic use only. Neither **C.2** nor **C.3** is used for cross-system ranking; they are solely used to inform **Panel B** (ours only) to show how strict scores shift when GT granularity/aliasing is accounted for.

G.6 Consolidated Implications (Header-Only CTA)

- **Strict baseline preserved.** All comparative conclusions in the paper rely on **official GT-strict** results (Panel A).
- **Quantified GT noise.** The audit suggests that ~**30.5%** of residual strict-GT “errors” reflect **GT-Refinement** (C.2) and ~**4%** reflect **Plausible Alternatives** (C.3) under header-only interpretation, lifting diagnostic Hit@1 from **0.45** to **≈0.76** (G.2) and **≈0.80** (C.2+C.3).
- **Benchmark design.** We recommend that future benchmarks distinguish **between header-only targets and context-dependent targets**, and maintain synonym/alias sets per entity/property to reduce avoidable mismatches.

G.7 Researcher-Proofing and Artifacts

- **Uncertainty reporting.** For our measures, we report **raw counts** and **95% Wilson CIs**; baseline CIs are omitted if not provided in the official report.
- **Reproducibility files.** We release an XLSX with one row per column:
SEMTAB_2024_dbpedia_match_results.xlsx [38]
- **Namespace coverage.** Some DBpedia concepts exist only as capitalised **classes** (e.g., dbo:Airport) without a lowercase property; systems restricted to properties may miss such cases. We index **both class and property** namespaces to mitigate false negatives.
- **Task alignment.** CTA is a **column-level** assignment task; benchmarks like **t2Dv2** provide **table-level** classes and are not directly comparable to column-level CTA outcomes.

Appendix H - HeadersIQ: A Metric for Data/Information Quality Monitoring

We introduce **HeadersIQ**, a simple yet effective metric that quantifies the overall data/information quality of a data source by reflecting the proportion of clean data items it contains.

The name **HeadersIQ** reflects its foundation in column **headers** and its focus on **Information Quality (IQ)**, while also evoking the idea of intelligent assessment, inspired by the concept of **Intelligence Quotient**.

Formal Definition.

Let:

N_c = number of columns,

N_r = number of rows,

$T = N_c \times N_r$ = total number of data items (cells),

$D = \{d_1, d_2, \dots, d_k\}$ = set of all **Data Quality Issues (DQIs)** evaluated (e.g., missing values, type mismatch, range violation, duplication, format inconsistency, etc.),

E_i = number of violations detected for DQI d_i .

Then the total number of detected issues is: $E = \sum_{i=1}^k E_i$

The **HeadersIQ** score is defined as:

$$\left(1 - \frac{E}{T}\right) \times 100$$

Normalization and Interpretation.

Because both E and T are measured at the cell level, the ratio E/T inherently normalizes by dataset size, allowing direct comparison across datasets.

A score of 100 indicates a fully clean dataset ($E = 0$); lower values represent the proportion of data items that violated the expected rules or formats.

Example:

For dataset 45 (303 rows, 14 columns $\rightarrow T=4242$) and $E=7$ total

Data Quality Issues (Errors):

$$\text{HeadersIQ} = (1 - 7/4242) \times 100 \approx 99.84$$

This indicates that 99.84% of all values in the dataset are clean, based on the identified issues.

Table 10 illustrates how HeadersIQ varies across the first ten datasets from UCI (including dataset 45), with values ranging from 93.14 to 100, providing a quick overview of the cleanliness levels.

Table K - HeadersIQ values for the first ten datasets from UCI

Dataset	Total Rows	Columns	Total Data Items	Total DQIs - Errors	HeadersIQ
52	150	5	750	0	100
45	303	14	4 242	7	99.84
2	32 561	15	488 415	4 262	99.13
107	178	14	2 492	170	93.18
42	214	11	2 354	0	100
58	20 000	17	340 000	0	100
142	10 000	21	210 000	1 000	95.24
92	4 601	58	266 858	0	100
103	435	17	7 395	392	94.7
27	690	16	11 040	757	93.14

HeadersIQ reflects the proportion of clean data items in each dataset, computed as $\left(1 - \frac{E}{T}\right) \times 100$, where E is the total number of detected issues, and T is the total number of data items. Implemented in the notebook *Attribute-BasedDataQualityAssessment.ipynb*[38].

This metric is especially valuable in real-world scenarios where hundreds of datasets may be evaluated over time. **HeadersIQ** supports scalable, automated data quality monitoring across data lakes and pipelines, providing clear signals for data governance teams and reducing manual inspection effort.

These results also demonstrate that **HeadersIQ** provides a dataset-level quality signal consistent with the **SemTab 2024 "IsGold?"** objective of assessing the overall reliability by automatically identifying structural noise, annotation inconsistencies, and NIL misuse.